Session 2 : Timers, Interrupts and DAC

1 Introduction

The informative parts are indicated by the icon , those requiring theoretical reflections are indicated by a are indicated by a symbol , and those requiring programming by the symbol .

1.1 Lab objectives

The purpose of this session is to continue to learn how to use the PSoC and its various features. More precisely, we will introduce the notion of time in our tasks using timers. We will see that there are two main ways to use timers : either by polling or by using an interrupt.
By the end of this session, you should be able to :

- Understand how a timer works.
- Implement a timer with appropriate frequency.
- Manage periodical tasks using timers.
- Use a timer by polling.
- Link a timer to an ISR (Interrupt Service Routine).
- Use the Digital-to-Analog Converter (DAC) e.g. by generating a sound on the audio output.

1.2 Prerequisites

- To know basic programming in C language (variables, arrays, functions, stream control, ect..)
- To have completed Lab 1.
- Here is a link towards PSoC tutorial videos :

(a) Tutorials

FIGURE 1 – Clickable QR code to tutorial videos.

- Here a link towards the wiki.

2 Timers

The purpose of this short assignment is to make the LEDs blink at a given frequency of 1 kHz. To do so, you will need a timer. In a µC, any temporal notion is derived from a component : the oscillator. It is an electronic circuit that produces a periodic signal called the clock (`CLK`). A timer is a hardware element which uses the clock to generate periodic signals whose frequency is a fraction of the clock frequency. For instance, the `CyDelay()` function allows to put the µC to sleep for precise time durations, but the programmer cannot know the execution time of other instructions. Hence, precise time management is impossible without timers. How does it work ? Here is link to the datasheet as well as a flowchart :

- A 8/16/24/32-bits register starts from a given value and is decremented at each `CLK` cycle.
- When this register reaches 0, the counter goes back to its initial value.
- At the same time, a specific bit named `TC` (in the `Timer_Status` register) goes to 1 to warn of overflow.
- It resumes a cycle only when the `Timer_Status` register value is read.

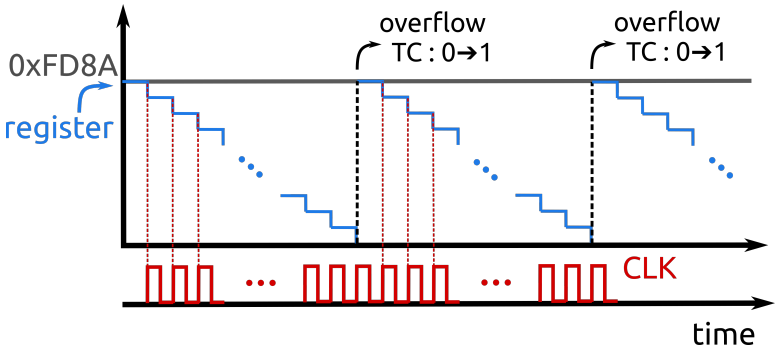

FIGURE 2 – Illustration of 16-bit timer operation.

Now that you have the working principle, we ask you to :

- Instantiate a 16-bit Timer in the `TopDesign.cysch`.

Question 1. Considering the default `CLK` at 24 MHz, what is the minimum frequency (max. period) that can be reached with a 16-bit timer ?

- Set the period to cause an overflow at a frequency of 1 kHz.

Question 2. Which value of `period` do you set for this ? Derive a general expression of `period` as function of the clock frequency and desired timer frequency.

- Add timer launch in your program.
- In the main loop, check by polling if the timer has overflowed using the following instruction :

```
overFlow = 0x80 & Timer_ReadStatusRegister();
```

Question 3. Explain in detail what this instruction does. What happens for the `TC` bit after this command ?

- Write a function that toggles the LEDs at 1 kHz.

> **Hint**
> Your eye won't be able to see a LED blinking at such frequency. Only a LED shining less brightly than in previous assignments.

- Modify your code to obtain a period of 1 s for the blinking of the LEDs.

Question 4. Find at least 2 ways to achieve this.

3 Digital-to-analog converter (DAC)

Hint

You will need earplugs to test this short assignment.

The purpose of this short assignment is to generate a tone on the jack audio output using a DAC. The principle of the DAC is the opposite of the ADC : the transformation of a sample coded on N bits to voltage in its output range. The obtained voltage is continuous in time (the output of the converter is maintained until the next sample, it's called sample and hold), but is always quantized because only 2^N different voltage values are achievable (each N bits binary code corresponds to one voltage). See Figure 3 for illustration.

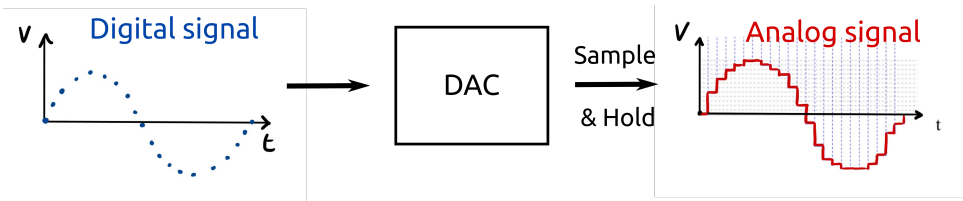

FIGURE 3 – Digital-to-analog conversion of samples which represent a sinusoidal wave.

Here, you are asked to realize a synthesizer capable of generating an arbitrary sound wave and convert it to sound through earplugs or speakers. In our case, we will simply generate a sine wave at adjustable frequency : the processor will send samples corresponding to the wanted waveform to a DAC and once rebuild, the signal will be send to the amplifier of the extension board [1]. You can then listen to generated sound by connecting your earplugs.

We propose the following steps to implement our synthesizer :

— In the initialization part of the `main.c`, fill a vector of length 100 containing the sound wave i.e. one full period of a sine wave defined between -1 and +1 but it should be defined as a global variable i.e. declared outside of the main() function.

Hint

To do so, you must :

— Include the file `math.h` in the header.

— Add the library to the linker : right click on the project << Build settings... << ARM GCC << Linker << General << Additional Libraries and add "m".

You'll then be able to call the function `sin()`, that takes as argument a floating number representing the angle (in rad) and that returns a floating number.

— Program a timer such that a sample of the sine wave is processed every 50 μs.

1. Element U3 on the extension board.

Question 5. What will be the corresponding sound frequency ? Derive a general expression for this.

- Instantiate a VDAC in the `TopDesign.cysch`.

> **Hint**
> The range is from 0 to 1.020 V and a slow speed is sufficient. Make sure you select "CPU" as the data source, since we will provide the data from the firmware. The strobe should be set to "Register Write", which means the DAC will provide a new analog value every time a new value is written in it's register.

- Instantiate an analog output pin and connect it to the VDAC. Do not forget to also connect it to the `DAC_OUT` pin of the extension board in the `Pins` tab (see schematics).
- In your `main.c`, poll the timer to detect overflows.
- Write the next value of the vector to DAC.

> **Hint**
> Find the appropriate function for that in the API of the VDAC component. What is it ?

Question 6. The DAC expect to see an 8-bit signal i.e. between 0 and 255. How would you change your sine wave such that it remains within this range at all times ?

Question 7. How do you think we could improve the sound quality ?

4 Interrupts

The purpose of this short assignment is to use a timer with a Interrupt Service Routine (ISR) instead of polling. As said in Section 1, it is the second main way to use a timer. An interrupt is a super-priority event. When it is latched, the firmware code in the `main.c` is temporarily interrupt while the ISR is executed. Note that an interrupt can be linked to any event e.g. overflow of a timer, the state of a GPIO, ect..
You can follow this tutorial for an introduction to interrupts :

(a) Interrupts

FIGURE 4 – Clickable QR code to Interrupt tutorial video.

We will try to modify the previous assignment on the DAC in order to send the data to the DAC based on Interrupt.

— Take your previous assigment on the DAC.

— In the `TopDesign.cysch`, instantiate an Interrupt (ISR) and connect it to the interrupt-output of the timer.

— Adapt your firmware so that your code that was previously inside your polling-loop is now your ISR, based on the introductory video. Do not forget to comment out the polling of the timer.

> **Hint**
> Read the documentation about the interrupt output in the timer datasheet more precisely in the "Run mode" section.

— In the `TopDesign.cysch`, check "on TC" in the Interrupt feature of the component.

Question 8. What is it for ?

— After an interrupt is triggered, the interrupt output remains asserted. Therefore, at the end of your ISR, you should call the function `Timer_ReadStatusRegister()`.

Question 9. Why should we still use this function ?